

0.13.2. Une lettre toutes les secondes, avec attendre — doublé, deux positions

Deux poser avant attendre : la même lettre en colonne 0 et en colonne 10.

1. Ce qu'il faut comprendre

C'est le 0.13 et le 0.13.1 réunis : deux `poser`, avant le même `attendre(1)`.

`attendre` **arrête tout** : les deux lettres sont posées, puis le programme attend une seconde, et recommence.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <attendre>` → [0.12.3. La fonction attendre\(\) — attendre des secondes](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **905 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      52 o ← la console, à l'allumage - la mise en route de la console
main()         48 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← AttendreSecondes - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreSecondes 17 o ← attendre() - attend des secondes entières
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles - la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte,
EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard,
EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :
poser()    × 2    24 o sur place, dans main()
attendre() × 1    5 o sur place, dans main() + AttendreSecondes (17 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <attendre>    // attend des secondes entières
```

2. UNE VARIABLE — LETTRE

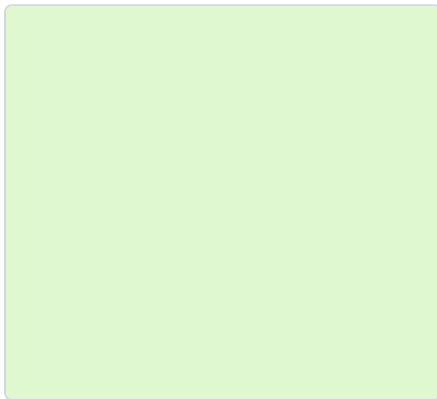
```
uint8_t lettre = 0;           // l'indice de la lettre affichée (0 = A)
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

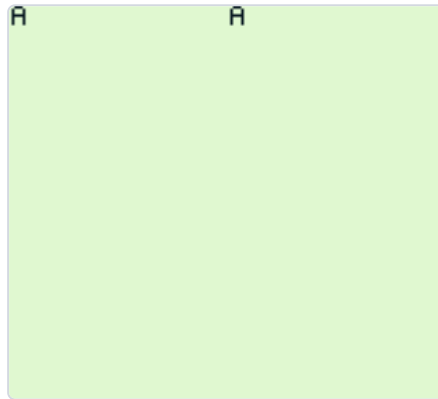
```
int main() {                                // Le programme commence ici.
    while (true) {                          // Pour toujours :
        poser(0, 0, ALPHABET[lettre]);      // colonne 0 (le 0.13)
        poser(10, 0, ALPHABET[lettre]);    // colonne 10 (le 0.13.1) : la même lettre
        attendre(1);                       // attend 1 seconde (tout s'arrête pendant ce temps)
        lettre++;                          // la lettre suivante
        if (lettre == sizeof(ALPHABET)) lettre = 0; // après Z (26 lettres), retour à A
    }
}
```

3. Ce qui se passe, étape par étape

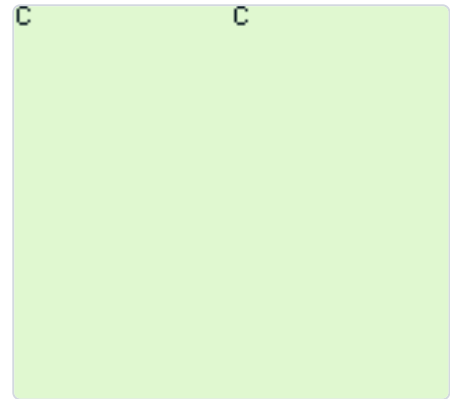
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



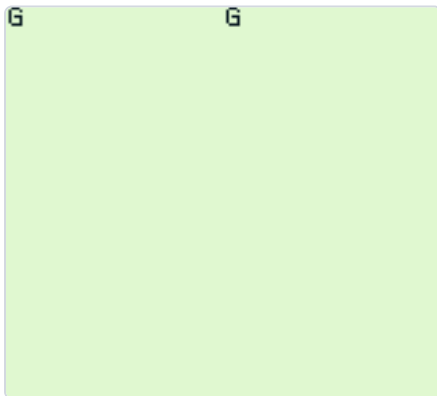
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

La même lettre en colonne 0 et en colonne 10, qui change chaque seconde.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la même lettre aux deux places

Ensuite : **0.14. Deux choses, deux rythmes** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++